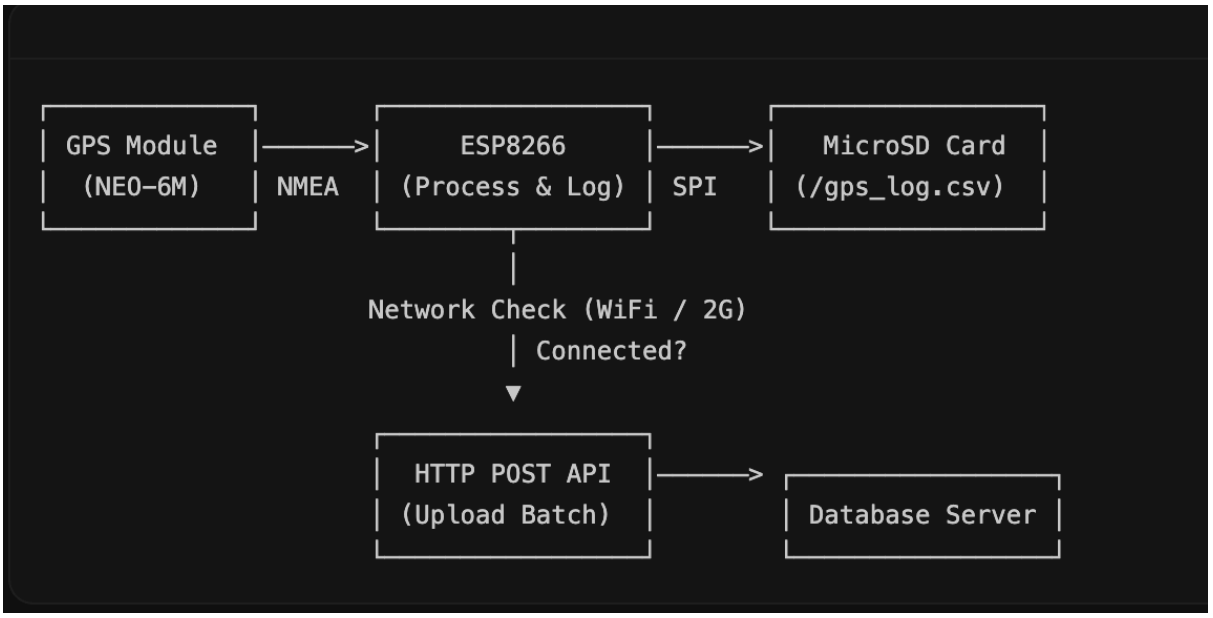
System Architecture




```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>
#include <SoftwareSerial.h>
#include <TinyGPS++.h>
#include <SPI.h>
#include <SD.h>
#include <ArduinoJson.h>

// --- Pin Definitions ---
#define GPS_RX_PIN 5  // D1
#define GPS_TX_PIN 4  // D2
#define SD_CS_PIN  15 // D8

// --- Wi-Fi Credentials ---
const char* WIFI_SSID = "YOUR_WIFI_SSID";
const char* WIFI_PASS = "YOUR_WIFI_PASSWORD";

// --- Remote Endpoint ---
const char* API_URL = "http://your-server-domain.com/api/gps/upload";

// --- Objects ---
SoftwareSerial gpsSerial(GPS_RX_PIN, GPS_TX_PIN);
TinyGPSPlus gps;

// --- Timing & Tracking ---
unsigned long lastLogTime = 0;
const unsigned long LOG_INTERVAL = 5000;   // Log to SD every 5 seconds
unsigned long lastSyncTime = 0;
const unsigned long SYNC_INTERVAL = 15000; // Check & sync every 15 seconds
```

```
// File paths
const char* LOG_FILE = "/gps_log.csv";
const char* POINTER_FILE = "/pointer.txt";

void setup() {
  Serial.begin(115200);
  gpsSerial.begin(9600);

  Serial.println("\n[INIT] Starting GPS & Storage Tracker...");

  // Initialize MicroSD Card
  if (!SD.begin(SD_CS_PIN)) {
    Serial.println("[ERROR] MicroSD Card Initialization Failed!");
  } else {
    Serial.println("[OK] MicroSD Card Initialized.");
  }

  // Connect Wi-Fi in station mode (non-blocking attempt)
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
}

void loop() {
  // 1. Process GPS serial stream continuously
  while (gpsSerial.available() > 0) {
    gps.encode(gpsSerial.read());
  }

  // 2. Log GPS data to SD Card at defined interval
  if (millis() - lastLogTime >= LOG_INTERVAL) {
    lastLogTime = millis();
    logGpsDataToSD();
  }

  // 3. Attempt sync if network is available
  if (millis() - lastSyncTime >= SYNC_INTERVAL) {
    lastSyncTime = millis();
    if (WiFi.status() == WL_CONNECTED) {
      uploadBufferedData();
    }
  }
}

// Write GPS point to SD Card CSV
void logGpsDataToSD() {
  if (!gps.location.isValid() || !gps.date.isValid() || !gps.time.isValid()) {
    Serial.println("[GPS] Searching for satellites fix...");
```

```
    return;
  }

  File file = SD.open(LOG_FILE, FILE_WRITE);
  if (file) {
    // ISO 8601 Timestamp format: YYYY-MM-DDTHH:MM:SSZ
    char isoTime[25];
    snprintf(isoTime, sizeof(isoTime), "%04d-%02d-%02dT%02d:%02d:%02dZ",
             gps.date.year(), gps.date.month(), gps.date.day(),
             gps.time.hour(), gps.time.minute(), gps.time.second());

    // Format: Timestamp,Latitude,Longitude,Speed,Satellites
    file.print(isoTime); file.print(",");
    file.print(gps.location.lat(), 6); file.print(",");
    file.print(gps.location.lng(), 6); file.print(",");
    file.print(gps.speed.kmph()); file.print(",");
    file.println(gps.satellites.value());

    file.close();
    Serial.printf("[SD LOG] Saved: %s | Lat: %.6f | Lng: %.6f\n", isoTime, gps.location.lat(),
gps.location.lng());
  } else {
    Serial.println("[ERROR] Failed to open CSV log file for writing.");
  }
}

// Read current offset pointer
uint32_t readPointer() {
  if (!SD.exists(POINTER_FILE)) return 0;
  File pFile = SD.open(POINTER_FILE, FILE_READ);
  if (!pFile) return 0;
  String val = pFile.readStringUntil('\n');
  pFile.close();
  return (uint32_t) val.toInt();
}

// Update offset pointer
void writePointer(uint32_t pos) {
  File pFile = SD.open(POINTER_FILE, FILE_WRITE);
  if (pFile) {
    pFile.seek(0);
    pFile.print(String(pos));
    pFile.println();
    pFile.close();
  }
}

// Upload unsent data batch
```

```
void uploadBufferedData() {
  if (!SD.exists(LOG_FILE)) return;

  uint32_t lastPos = readPointer();
  File file = SD.open(LOG_FILE, FILE_READ);
  if (!file) return;

  uint32_t fileSize = file.size();
  if (lastPos >= fileSize) {
    file.close(); // No new data to sync
    return;
  }

  // Seek to last unsent position
  file.seek(lastPos);

  // Read up to 20 lines into JSON payload batch
  StaticJsonDocument<2048> doc;
  JsonArray array = doc.to<JsonArray>();

  int linesRead = 0;
  uint32_t nextPos = lastPos;

  while (file.available() && linesRead < 20) {
    String line = file.readStringUntil('\n');
    line.trim();
    if (line.length() == 0) continue;

    // Parse CSV line
    int c1 = line.indexOf(',');
    int c2 = line.indexOf(',', c1 + 1);
    int c3 = line.indexOf(',', c2 + 1);
    int c4 = line.indexOf(',', c3 + 1);

    if (c1 > 0 && c2 > 0 && c3 > 0) {
      JsonObject obj = array.createNestedObject();
      obj["timestamp"]  = line.substring(0, c1);
      obj["lat"]        = line.substring(c1 + 1, c2).toDouble();
      obj["lng"]        = line.substring(c2 + 1, c3).toDouble();
      obj["speed_kmh"]  = line.substring(c3 + 1, c4).toFloat();
      obj["satellites"] = line.substring(c4 + 1).toInt();
      linesRead++;
    }
    nextPos = file.position();
  }
  file.close();

  if (linesRead == 0) return;
```

```
  // Send HTTP POST payload
  String jsonString;
  serializeJson(doc, jsonString);

  WiFiClient client;
  HTTPClient http;
  http.begin(client, API_URL);
  http.addHeader("Content-Type", "application/json");

  int httpCode = http.POST(jsonString);
  if (httpCode == HTTP_CODE_OK || httpCode == HTTP_CODE_CREATED) {
    Serial.printf("[SYNC SUCCESS] Uploaded %d records to database.\n", linesRead);
    writePointer(nextPos); // Save updated read position
  } else {
    Serial.printf("[SYNC ERROR] HTTP POST failed with code: %d\n", httpCode);
  }
  http.end();
}
```